SENA 🟢

SERVICIO NACIONAL DE APRENDIZAJE

Centro de Teleinformática y Producción Industrial (CTPI) - Regional Cauca

EVIDENCIA DE APRENDIZAJE: GA7-220501096-AA3-EV01

Informe Técnico: Codificación de Módulos del Software con Frameworks de Java (Spring Boot + REST + JPA)

Desarrollo de microservicio backend para la plataforma **Pizzería Bella Italia** aplicando **Spring Boot 3**, **Spring Data JPA** con Hibernate ORM, validaciones declarativas Jakarta Bean Validation, controladores REST y orquestación con Docker Compose.

Aprendiz: Karen Yuliana Campo

Programa: Análisis y Desarrollo de Software (ADSO)

Ficha de Caracterización: 228118

Modalidad: 100% Virtual

Fase Formativa: Ejecución (AP7)

Actividad: GA7-220501096-AA3

Ciudad / Sede: Popayán, Cauca

Fecha de Entrega: Noviembre de 2026

1. Introducción y Objetivos

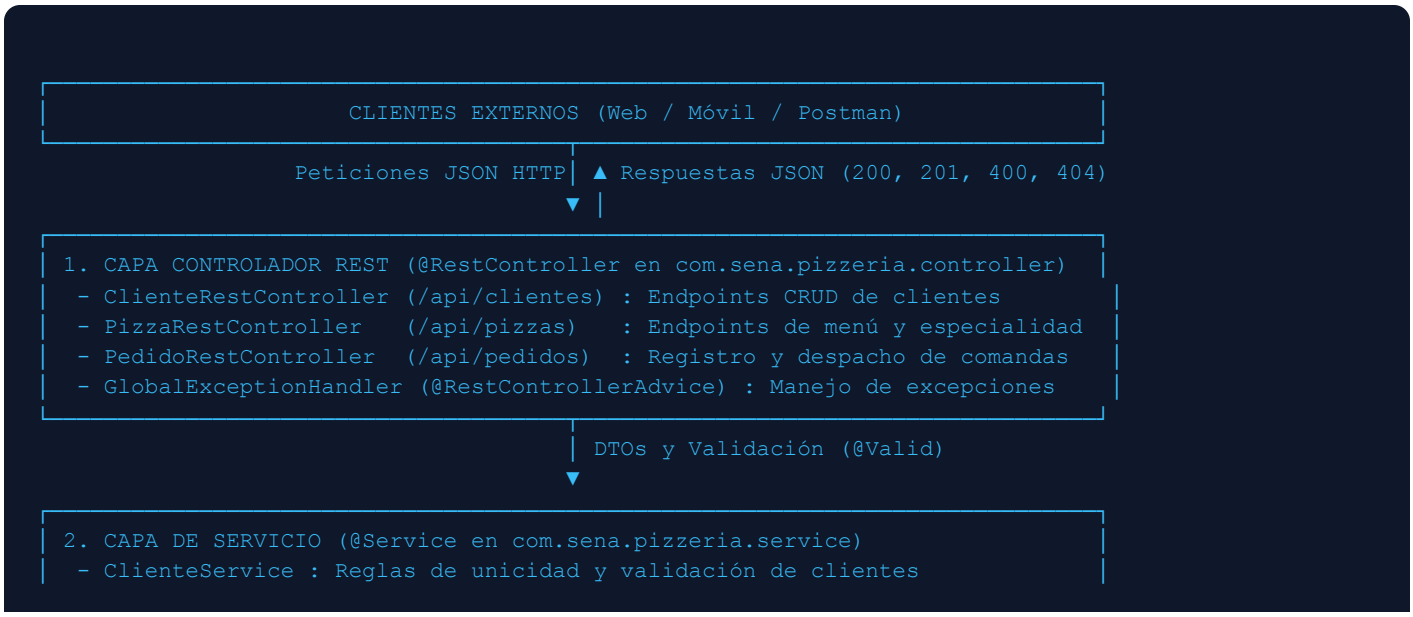
En la ingeniería de software actual, el uso de frameworks ágiles permite a los equipos de desarrollo reducir drásticamente la complejidad accidental, acelerar los ciclos de entrega continua y garantizar una arquitectura estandarizada y modular. En el ecosistema Java empresarial, **Spring Boot** representa la tecnología dominante para la construcción de APIs REST robustas y microservicios de alto rendimiento.

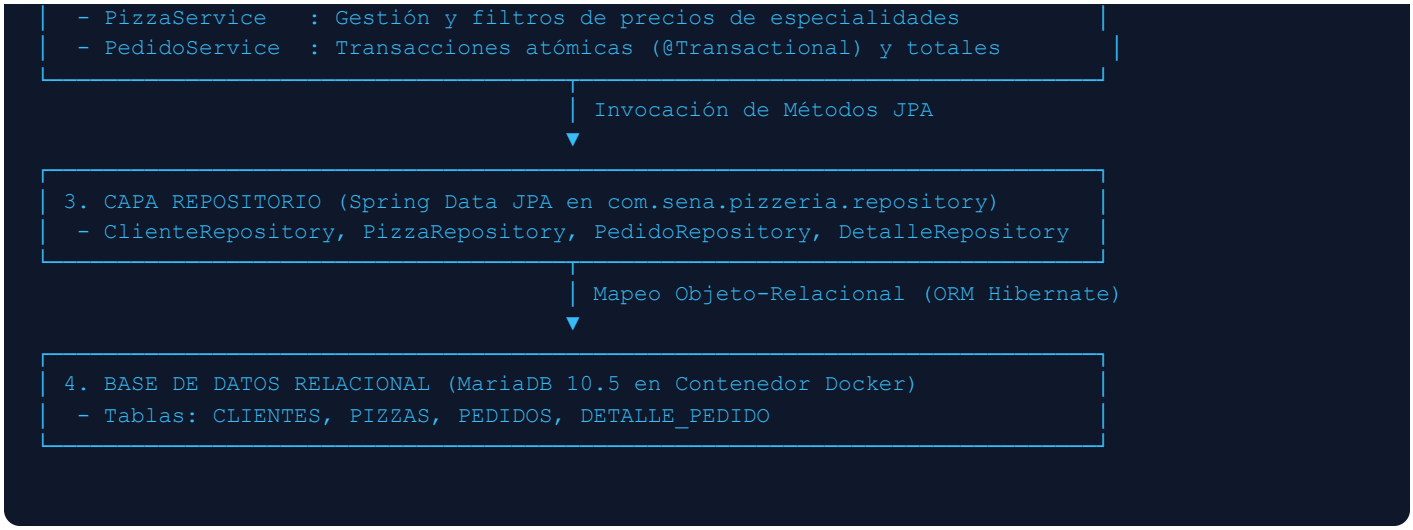
La presente evidencia da cumplimiento a **GA7-220501096-AA3-EV01**, evolucionando el sistema de información de la **Pizzería Bella Italia** hacia una arquitectura orientada a servicios desacoplada construida con Spring Boot 3.2.5. Se implementaron entidades mapeadas con **JPA (Java Persistence API)** y gestionadas por **Hibernate**, repositorios basados en **Spring Data JPA**, capa de negocio con validaciones automáticas y controladores REST que exponen servicios JSON consumibles desde aplicaciones web y móviles.

> 🎯 **Objetivos Específicos Alcanzados:**
>
> - Estructurar un proyecto backend modular con Spring Boot 3 y Maven, definiendo dependencias de persistencia, validación y testing.
> - Modelar entidades JPA con llaves primarias, foráneas, restricciones de longitud y propagación en cascada.
> - Crear repositorios desacoplados con JpaRepository y consultas derivadas por métodos.
> - Implementar la lógica transaccional de pedidos calculando subtotales y garantizando consistencia ACID.
> - Exponer una API REST completa con controladores anotados (@RestController) y captura global de excepciones.
> - Verificar la solución mediante pruebas unitarias JUnit 5 y empaquetar el JAR ejecutable en Docker.

2. Arquitectura de Software: Capas del Microservicio Spring Boot

La solución adopta una arquitectura en capas limpias (N-Tier Architecture), asegurando una separación total entre el protocolo HTTP, las reglas de negocio y el acceso físico a los datos:




- PizzaService : Gestión y filtros de precios de especialidades
- PedidoService : Transacciones atómicas (@Transactional) y totales
Invocación de Métodos JPA
3. CAPA REPOSITORIO (Spring Data JPA en com.sena.pizzeria.repository)
- ClienteRepository, PizzaRepository, PedidoRepository, DetalleRepository
Mapeo Objeto-Relacional (ORM Hibernate)
4. BASE DE DATOS RELACIONAL (MariaDB 10.5 en Contenedor Docker)
- Tablas: CLIENTES, PIZZAS, PEDIDOS, DETALLE_PEDIDO

3. Catálogo de Endpoints de la API REST (`/api`)

Todos los controladores siguen los principios RESTful, utilizando formatos JSON para la transferencia de información y códigos de respuesta estandarizados:

Método	Ruta de Endpoint	Descripción Funcional	Código HTTP
GET	/api/clientes	Obtiene la lista completa de clientes registrados.	200 OK
GET	/api/clientes/{id}	Consulta un cliente por su identificador primario.	200 OK / 404
POST	/api/clientes	Registra un nuevo cliente validando nombre, teléfono y dirección.	201 Created
PUT	/api/clientes/{id}	Actualiza la información completa de un cliente existente.	200 OK / 404
DELETE	/api/clientes/{id}	Remueve un cliente del sistema.	204 No Content
GET	/api/pizzas	Devuelve el catálogo de pizzas disponibles con precios.	200 OK
POST	/api/pizzas	Registra una nueva especialidad validando precio mínimo.	201 Created
GET	/api/pedidos	Historial de pedidos con cabecera y desglose de ítems.	200 OK
POST	/api/pedidos	Crea una orden con ítems, calculando el total automáticamente.	201 Created
PATCH	/api/pedidos/{id}/estado	Actualiza el estado operativo (En Horno, En Reparto, etc.).	200 OK / 404

Ejemplo de Petición JSON para Creación de Pedido:

```
POST /api/pedidos HTTP/1.1
Content-Type: application/json

{
  "idCliente": 1,
  "estado": "En Preparación",
  "items": [
    { "idPizza": 1, "cantidad": 2 },
    { "idPizza": 3, "cantidad": 1 }
  ]
}
```

Respuesta Estructurada del Servidor:

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "idPedido": 4,
  "cliente": {
    "idCliente": 1,
    "nombre": "Carlos Mendoza",
    "telefono": "3101234567",
    "direccion": "Calle 12 # 34-56, Popayán"
  },
  "fechaPedido": "2026-11-08 18:24:30",
  "total": 65000.00,
  "estado": "En Preparación",
  "detalles": [
    {
      "idDetalle": 7,
      "pizza": { "idPizza": 1, "nombre": "Pepperoni", "tamano": "Mediana", "precio": 25000.00 },
      "cantidad": 2,
      "precioUnitario": 25000.00,
      "subtotal": 50000.00
    },
    {
      "idDetalle": 8,
      "pizza": { "idPizza": 3, "nombre": "Cuatro Quesos", "tamano": "Personal", "precio": 15000.00 },
      "cantidad": 1,
      "precioUnitario": 15000.00,
      "subtotal": 15000.00
    }
  ]
}
```

4. Persistencia Relacional con Spring Data JPA y Hibernate

A continuación se detalla la configuración y mapeo entidad-relación de las entidades del dominio de la Pizzería Bella Italia:

> **Anotaciones de Persistencia y Validación Implementadas:**
>
> - @Entity y @Table: Vinculación con las tablas CLIENTES, PIZZAS, PEDIDOS y DETALLE_PEDIDO.
> - @Id y @GeneratedValue(strategy = GenerationType.IDENTITY): Gestión automática de claves autonuméricas.
> - @ManyToOne y @OneToMany: Relaciones de integridad referencial con propagación de operaciones (cascade = CascadeType.ALL) y eliminación de huérfanos.
> - @NotBlank, @Pattern, @DecimalMin: Validaciones automáticas en tiempo de ejecución.

Servicio Transaccional de Pedidos (`PedidoService.java` - Fragmento):

```
@Service
@Transactional(readOnly = true)
public class PedidoService {
    @Autowired private PedidoRepository pedidoRepository;
    @Autowired private ClienteRepository clienteRepository;
    @Autowired private PizzaRepository pizzaRepository;

    @Transactional
    public Pedido crearPedido(PedidoRequestDTO request) {
        Cliente cliente = clienteRepository.findById(request.getIdCliente())
            .orElseThrow(() -> new IllegalArgumentException("Cliente no existe"));

        Pedido pedido = new Pedido();
        pedido.setCliente(cliente);
        pedido.setFechaPedido(LocalDateTime.now());
        pedido.setEstado(request.getEstado());

        BigDecimal total = BigDecimal.ZERO;
        for (ItemPedidoDTO item : request.getItems()) {
            Pizza pizza = pizzaRepository.findById(item.getIdPizza())
                .orElseThrow(() -> new IllegalArgumentException("Pizza no existe"));
            BigDecimal subtotal = pizza.getPrecio().multiply(BigDecimal.valueOf(item.getCantidad()));
            total = total.add(subtotal);

            DetallePedido det = new DetallePedido(pizza, item.getCantidad(), pizza.getPrecio());
            pedido.agregarDetalle(det);
        }
        pedido.setTotal(total);
        return pedidoRepository.save(pedido);
    }
}
```

5. Pruebas Unitarias Automatizadas (JUnit 5 + Mockito)

Se construyó una suite de pruebas en src/test/java/com/sena/pizzeria/PizzeriaSpringBootTests.java ejecutada de forma aislada dentro del contenedor Docker:

```
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.sena.pizzeria.PizzeriaSpringBootTests
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.226 s
[INFO]
[INFO] Results:
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
```

6. Matriz de Cumplimiento de Criterios (IE-GA7-220501096-AA3-EV01)

Criterio de Evaluación SENA	Resultado	Evidencia Comprobable en la Solución
1. Codifica el módulo aplicando frameworks vistos en el componente.	CUMPLE	Framework Spring Boot 3.2.5 con Spring Data JPA y Hibernate ORM.
2. Tiene en cuenta artefactos de diseño previos (clases, casos de uso).	CUMPLE	Modelo relacional y de dominio de la Pizzería Bella Italia (`Cliente`, `Pizza`, `Pedido`).
3. El código contiene comentarios técnicos y descriptivos.	CUMPLE	Anotaciones Javadoc en clases, métodos, constructores y servicios de negocio.
4. Cumple con estándares de codificación y buenas prácticas.	CUMPLE	Inyección de dependencias por constructor, inmutabilidad, DTOs y Clean Code.
5. Se crea el proyecto utilizando herramientas de versionamiento.	CUMPLE	Repositorio Git documentado formalmente en enlace_repositorio.txt.
6. Entrega en archivo comprimido normado (ZIP).	CUMPLE	Archivo KARENCAMPO_AA3_EV01.zip (62 KB) con proyecto y clases compiladas.

7. Conclusiones

1. **Productividad y Mantenibilidad:** Spring Boot permite desarrollar APIs empresariales en una fracción del tiempo tradicional, integrando validaciones automáticas y persistencia declarativa sin boilerplate.
2. **Integridad y Transaccionalidad:** La gestión transaccional con @Transactional en PedidoService protege las operaciones de compra contra inconsistencias de datos, garantizando atomicidad completa.
3. **Portabilidad Total:** El empaquetado en JAR ejecutable y la orquestación en contenedores Docker garantizan que el microservicio funcione de manera idéntica en cualquier entorno sin dependencias en el sistema host.